MIT 8103 / 2026-2027

Portfolio 3: transactions and concurrency

A transaction groups changes so they can succeed together or be undone together. Use the exercise database only. Do not leave a transaction open on a shared production server.

1. Commit a complete sale

```
USE GreenBasketDB;
GO

-- Run once, after PDF 08 is finished.
SET XACT_ABORT ON;
BEGIN TRANSACTION;

UPDATE dbo.Products
SET StockQuantity = StockQuantity - 1
WHERE ProductID = 1 AND StockQuantity >= 1;

IF @@ROWCOUNT = 1
BEGIN
 INSERT INTO dbo.Orders (OrderID, CustomerID, OrderDate)
VALUES (4, 1, '20260904');

    INSERT INTO dbo.OrderItems
        (OrderID, ProductID, Quantity, UnitPrice)
    SELECT 4, ProductID, 1, Price
    FROM dbo.Products
    WHERE ProductID = 1;

 COMMIT TRANSACTION;
 PRINT 'Sale saved.';
END
ELSE
BEGIN
 ROLLBACK TRANSACTION;
 PRINT 'Sale not saved: product missing or insufficient stock.';
END;
GO

SELECT ProductID, StockQuantity
FROM dbo.Products WHERE ProductID = 1;
SELECT * FROM dbo.Orders WHERE OrderID = 4;
SELECT * FROM dbo.OrderItems WHERE OrderID = 4;
```

Expected after a fresh run: rice stock changes from 20 to 19; order 4 belongs to customer 1; its line is one rice bag at NGN 25,000. COMMIT saves all three changes. The stock update uses StockQuantity - 1 inside SQL Server, avoiding the unsafe pattern of reading a balance into an application and later overwriting it.

@@ROWCOUNT immediately checks whether the update affected one row. The stock condition prevents selling unavailable stock. SET XACT_ABORT ON makes most runtime errors abort and roll back the transaction. Explicit IDs keep the lesson simple; do not rerun this sale with OrderID 4 already present. Identity-generated IDs would be preferable in a full application.

2. Roll back a trial change

```
USE GreenBasketDB;
GO

BEGIN TRANSACTION;

UPDATE dbo.Products
SET StockQuantity = StockQuantity - 2
WHERE ProductID = 2 AND StockQuantity >= 2;

SELECT StockQuantity AS DuringTransaction
FROM dbo.Products WHERE ProductID = 2;

ROLLBACK TRANSACTION;

SELECT StockQuantity AS AfterRollback
FROM dbo.Products WHERE ProductID = 2;
```

Expected: beans stock is 28 inside the transaction, then 30 after ROLLBACK. The change was cancelled. This example can be repeated because it does not keep the change.

ACID in this example

Atomicity: stock, order and line are saved together. **Consistency:** keys and checks keep valid relationships and quantities. **Isolation:** locks coordinate competing operations. **Durability:** after COMMIT succeeds, SQL Server preserves the committed change through normal recovery.

3. Two-window concurrency demonstration

The problem is a non-repeatable read: another person changes a price between two reads. REPEATABLE READ keeps the selected row protected until the reading transaction ends. A competing price update must wait. It does not prevent every possible concurrency issue, such as all phantom rows.

Step A: Open two separate SSMS query windows connected to GreenBasketDB. Run the following in Window A only:

```
USE GreenBasketDB;
GO

SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
BEGIN TRANSACTION;

SELECT Price FROM dbo.Products WHERE ProductID = 3;
-- Leave this transaction open briefly. Now run Window B.
```

Step B: Run this in Window B. It should wait at the UPDATE while A remains open; waiting here is the demonstration, not a failure.

```
USE GreenBasketDB;
GO

SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
UPDATE dbo.Products
SET Price = 8500.00
WHERE ProductID = 3;

SELECT Price FROM dbo.Products WHERE ProductID = 3;
```

Step C: Return to the original Window A and run this. Its second read should still be 8000.00. COMMIT releases the lock and B can finish.

```
USE GreenBasketDB;
GO

-- Run in the SAME Window A connection, not a new window.
SELECT Price FROM dbo.Products WHERE ProductID = 3;
COMMIT TRANSACTION;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

Step D: Window B should now show 8500.00. After it finishes, restore the original price:

```
USE GreenBasketDB;
GO

-- Run after both windows have completed.
UPDATE dbo.Products SET Price = 8000.00 WHERE ProductID = 3;
SELECT ProductID, Price FROM dbo.Products WHERE ProductID = 3;
```

Evidence and cleanup

Capture A's first read, B waiting, A's unchanged second read and B's final result. Record the sequence in your README. If you stop halfway, execute IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION; in the original A connection to release its transaction, then allow B to finish and restore the price.

Under ordinary READ COMMITTED with no spanning read lock, the second read could see a committed changed price. This example uses REPEATABLE READ specifically to show the control mechanism. It does not need NOLOCK, cursors, triggers or complex locking hints.